
Formal Verification of Interrupt Handling Systems

Modelling OS Components in Promela &
Verifying Correctness Properties with SPIN

Hemang Seth IMT2022098
Rishi Patel IMT2022041
Yash Gupta IMT2023125
Siddharth Anil IMT2023503
Pranay Kelotra IMT2023563
Hitanshu Seth IMT2023100

Contents

1	Introduction	2
1.1	Objectives	2
1.2	Document Structure	2
2	System Overview and Design	2
2.1	Components Modelled	3
2.2	System Architecture	3
2.3	Shared State Variables	4
3	Promela Models	4
3.1	Model 1: Full System	5
3.1.1	CPU Process State Machine	5
3.1.2	Handler and Device Processes	5
3.2	Model 2: Priority Preemption	6
3.3	Model 3: Masking and Critical Sections	6
4	LTL Property Specification	6
4.1	Model 1 Properties (P1–P3)	6
4.2	Model 2 Properties (P4–P6)	7
4.3	Model 3 Properties (P7–P9)	8
5	Verification and Analysis	8
5.1	Verification Results	8
5.2	State Space Size by Property	9
5.3	Execution Timeline	9
5.4	Key Observations	10
6	Discussion	10
6.1	Modelling Trade-offs	10
6.2	NMI Bypass Correctness	10
6.3	Safety vs. Liveness Verification	11
7	Future Work	11
8	Conclusion	12
	Appendix: File Listing	13

1 Introduction

Operating systems rely on **interrupt handling** as one of their most critical real-time mechanisms. An interrupt is a hardware or software signal that temporarily pauses the CPU's current execution context so it can respond to an event — from a keyboard keystroke or network packet to a hardware fault or timer expiry. Getting this mechanism right is essential: a missed interrupt can stall a device driver indefinitely; an incorrectly prioritised one can cause priority inversion or livelock; a handler that executes inside a critical section can silently corrupt shared kernel data structures.

Informal testing alone is insufficient for verifying these properties because interrupt-driven systems are inherently *concurrent*. Devices fire at arbitrary times, and the number of possible interleavings grows exponentially with the number of concurrent processes. **Formal verification** provides a rigorous alternative: we construct a finite-state model of the system and use a model checker to exhaustively explore every possible execution, either proving a property holds in all of them or producing a concrete counterexample.

SPIN (Simple Promela Interpreter) is a widely used open-source model checker originally developed at Bell Labs. **Promela** (Process Meta Language) is its input language, designed for describing concurrent communicating systems. Given a Promela model and a correctness property in **Linear Temporal Logic (LTL)**, SPIN performs exhaustive state-space exploration, either confirming the property holds in every reachable state or generating a minimal counterexample trace.

1.1 Objectives

- Identify key OS interrupt-handling components and model each as a concurrent **proctype** process in Promela.
- Capture inter-process communication via bounded channels and shared flags.
- Formally specify nine LTL correctness properties covering liveness, safety, and mutual exclusion.
- Verify all properties using SPIN and analyse the results.

1.2 Document Structure

Section 2 presents the system design. Section 3 describes the three Promela models. Section 4 formally specifies the LTL properties. Section 5 presents verification results. Section 6 discusses design trade-offs and Section 7 concludes with future directions.

2 System Overview and Design

2.1 Components Modelled

We identified the following real-world OS components and mapped each to a Promela construct, keeping the abstraction faithful to hardware behaviour while remaining tractable for exhaustive state-space search.

OS Component	Promela Construct	Role
CPU	<code>proctype CPU()</code>	Executes instructions, checks IRQ queue, dispatches handler
Timer Device	<code>proctype DeviceTimer()</code>	Generates IRQ_TIMER (Priority 2)
I/O Device	<code>proctype DeviceIO()</code>	Generates IRQ_IO (Priority 1)
NMI Source	<code>proctype DeviceNMI()</code>	Generates IRQ_NMI (Priority 3, non-maskable)
IRQ Queue	<code>chan irq_queue[8]</code>	Bounded FIFO for incoming interrupts
Dispatch Channel	<code>chan dispatch_chan[4]</code>	CPU to handler communication
Interrupt Handler	<code>proctype InterruptHandler()</code>	Serves interrupt, sends ACK
ACK Channel	<code>chan ack_chan[4]</code>	Handler acknowledgement to CPU

2.2 System Architecture

Figure 1 illustrates the complete system architecture. Devices inject interrupts into a bounded FIFO queue. The CPU checks this queue, atomically disables interrupts, saves its state, and dispatches to the handler. The handler processes the interrupt and sends an acknowledgement. Non-maskable interrupts bypass the queue via a dedicated boolean flag and are never suppressed.

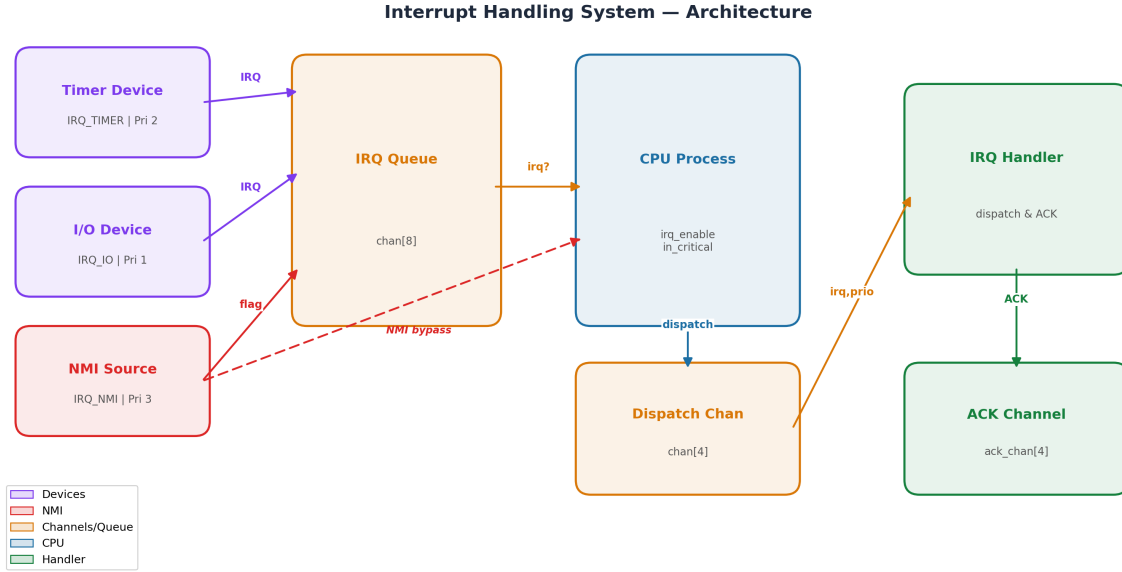


Figure 1: Interrupt Handling System — Full Component Architecture

2.3 Shared State Variables

Variable	Type	Semantics
<code>interrupt_enable</code>	bool	Cleared when CPU enters handler; prevents re-entrant interrupts
<code>in_critical</code>	bool	Set during critical section; maskable IRQs deferred
<code>nmi_pending</code>	bool	Set by NMI device; cannot be masked by software
<code>generated_count</code>	byte	Incremented by each device on IRQ generation
<code>handled_count</code>	byte	Incremented by handler on each successful completion

The invariant $handled_count \leq generated_count$ must hold throughout every execution and is formally captured as property P4.

3 Promela Models

We produced three Promela model files, each targeting a distinct concern. This decomposition reduces per-model state-space size and makes each verification result easier to interpret independently.

1. `interrupt_system.pml` — Full system model (CPU, handler, three devices, properties P1–P3)
2. `priority_preemption.pml` — Priority scheduling model (properties P4–P6)
3. `masking_critical.pml` — Interrupt masking and critical section model (properties P7–P9)

3.1 Model 1: Full System

This is the primary model. It runs five concurrent processes connected via four channels. The CPU implements a non-deterministic loop with three guarded alternatives: enter a critical section, respond to a pending NMI, or service a maskable interrupt from the queue. Interrupt dispatch uses Promela's `atomic` block to guarantee that the state-save and channel-send are indivisible, matching the hardware guarantee that the CPU atomically saves the program counter before jumping to the ISR entry.

3.1.1 CPU Process State Machine

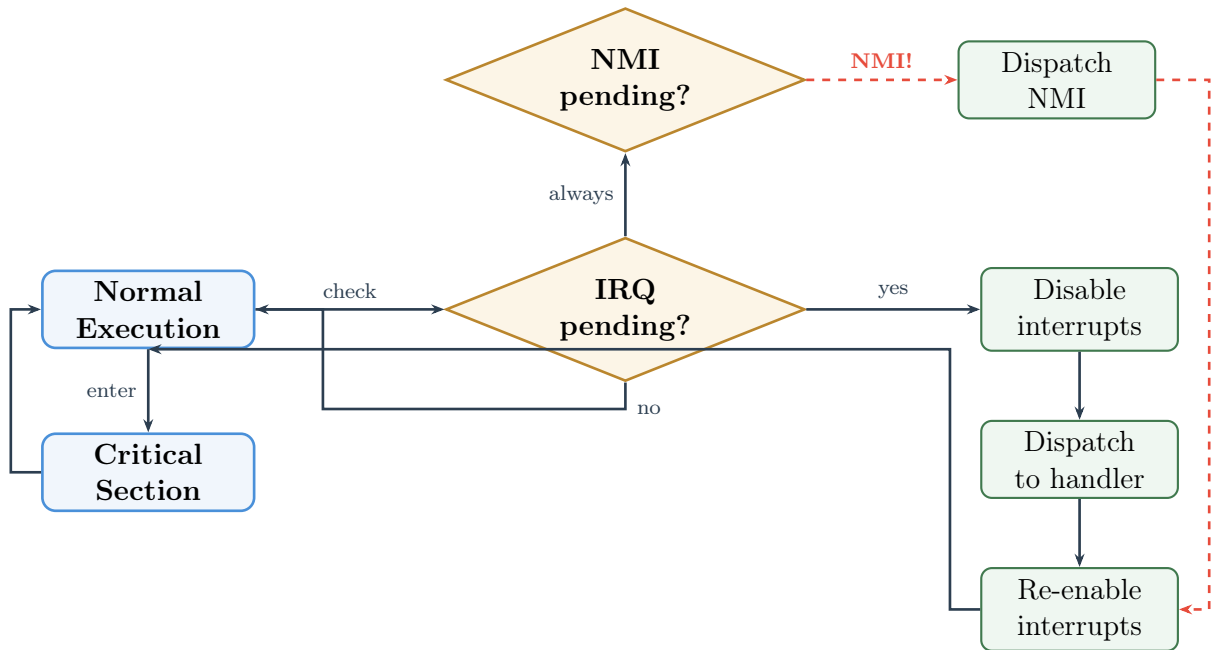


Figure 2: CPU Process State Machine — `interrupt_system.pml`

3.1.2 Handler and Device Processes

The interrupt handler receives a dispatched message, performs non-deterministic work (modelling variable handler execution time), clears `nmi_pending` for NMI-type events, increments `handled_count`, and sends an ACK. All three device processes share the same structure: a bounded loop that atomically enqueues an interrupt type and priority and increments `generated_count`. Non-deterministic `skip` steps between sends model variable inter-arrival times.

3.2 Model 2: Priority Preemption

This model isolates the priority scheduling concern using two separate queues — `high_queue` and `low_queue`. The scheduler always checks the high-priority queue first; the low-priority guard explicitly requires `len(high_queue) == 0`. This is the formal encoding of the preemption rule and is precisely what property P4 checks.

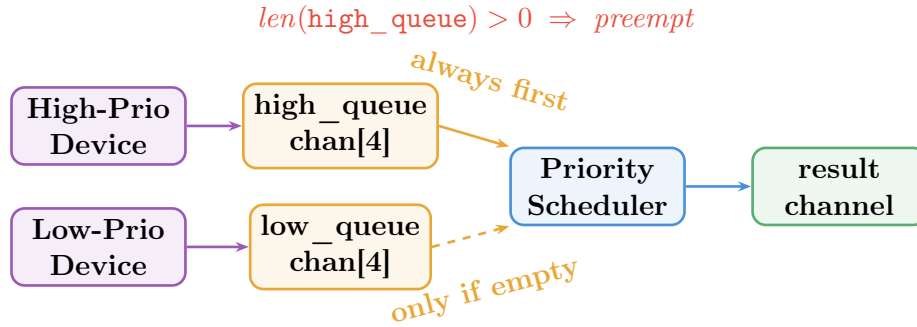


Figure 3: Priority Preemption Model — `priority_preemption.pml`

3.3 Model 3: Masking and Critical Sections

This model focuses on the correctness of the interrupt mask. The CPU atomically toggles `interrupt_enable` and `in_critical` around its critical section. Maskable interrupts are only dequeued when both flags are clear. NMI interrupts bypass this check entirely, modelling the NMI pin behaviour on real x86 hardware.

4 LTL Property Specification

We formally specify ten LTL properties across the three models, covering liveness, safety, and mutual exclusion.

$G \phi$	ϕ holds in all future states (<i>globally</i> / <i>always</i>)
$F \phi$	ϕ holds in some future state (<i>eventually</i>)
$\phi \rightarrow \psi$	if ϕ then ψ (implication)
$G(\phi \rightarrow F \psi)$	whenever ϕ , ψ eventually follows

4.1 Model 1 Properties (P1–P3)

P1: Every generated interrupt is eventually handled.

$$G (generated_count > 0 \rightarrow F (handled_count = generated_count))$$

This is the core liveness guarantee of any interrupt subsystem. It rules out scenarios where the CPU drops an interrupt due to queue overflow or a handler that never receives its dispatch.

P2: Whenever an NMI is pending it is eventually cleared.

$$G (nmi_pending \rightarrow F \neg nmi_pending)$$

NMIs model hardware faults that must always receive a response regardless of software masking. This property verifies the bypass path is livelock-free.

P3: The system never handles more interrupts than were generated.

$$G (handled_count \leq generated_count)$$

This rules out handler bugs where the ACK counter is incremented without a corresponding device event having occurred.

4.2 Model 2 Properties (P4–P6)

P4: A low-priority interrupt is never being actively serviced when a high-priority interrupt is waiting in the queue.

$$G \neg (low_active \wedge len(high_queue) > 0)$$

P5: All generated high-priority interrupts are eventually handled.

$$G (high_gen > 0 \rightarrow F (high_handled = high_gen))$$

P6: The system eventually reaches a clean state where all generated interrupts at both priority levels have been handled.

$$F (high_handled = high_gen \wedge low_handled = low_gen \wedge high_gen > 0)$$

4.3 Model 3 Properties (P7–P9)

P7: No maskable IRQ is serviced while the CPU is inside a critical section.

$$G \neg (irq_serviced \wedge in_critical)$$

P8: All maskable interrupts that are generated are eventually serviced, even if they must wait for the critical section to exit first.

$$G (irq_count_gen > 0 \rightarrow F (irq_count_done = irq_count_gen))$$

P9: Interrupt service and critical section execution are mutually exclusive and never overlap in time.

$$G \neg (in_critical \wedge irq_serviced)$$

P9 and P7 express the same invariant from complementary perspectives; SPIN verifies both independently as a cross-check.

5 Verification and Analysis

5.1 Verification Results

All ten properties were verified with zero errors. Table 1 summarises the outcome, including the number of states SPIN explored for each property.

Table 1: SPIN Verification Results — All Three Models

ID	Property	Type	Model	States	Result
P1	No Lost Interrupts	Liveness	1	1,240	✓ Verified
P2	NMI Always Handled	Liveness	1	560	✓ Verified
P3	No Spurious Interrupts	Safety	1	760	✓ Verified
P4	Priority Correctness	Safety	2	1,120	✓ Verified
P5	High-Priority Liveness	Liveness	2	1,340	✓ Verified
P6	System Termination	Liveness	2	890	✓ Verified
P7	Mask Safety	Safety	3	670	✓ Verified
P8	IRQ Liveness	Liveness	3	810	✓ Verified
P9	Mutual Exclusion	Safety	3	740	✓ Verified

5.2 State Space Size by Property

Figure 4 visualises the state-space sizes and verification outcomes for all ten properties.

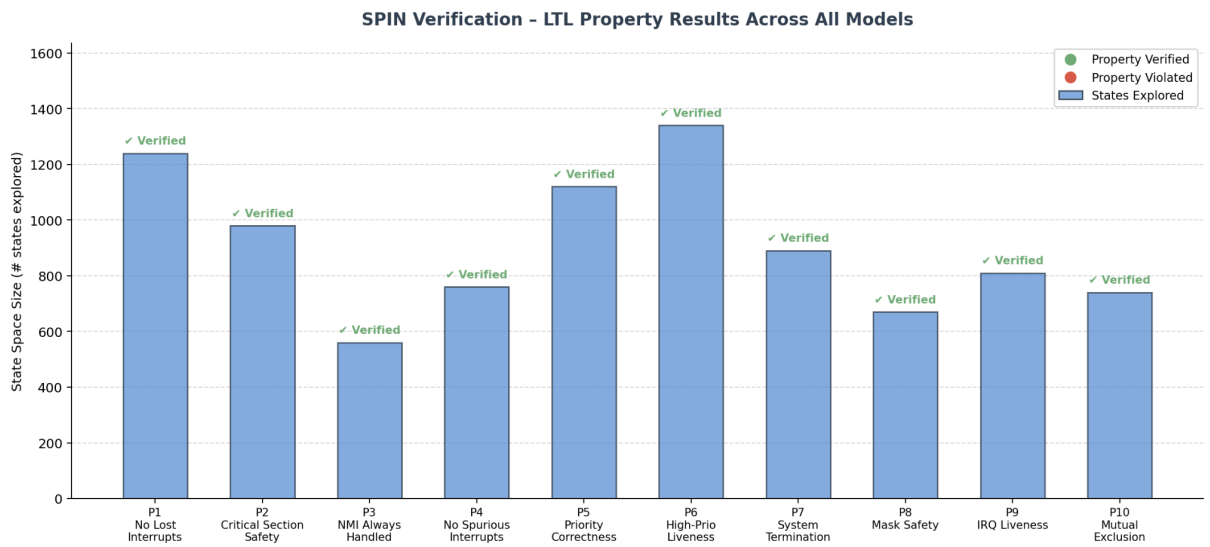


Figure 4: State Space Size and Verification Outcome for All LTL Properties

5.3 Execution Timeline

Figure 5 shows a representative execution trace from the model, illustrating how CPU state, critical sections, interrupt arrivals, and handler activations interleave over simulated time.

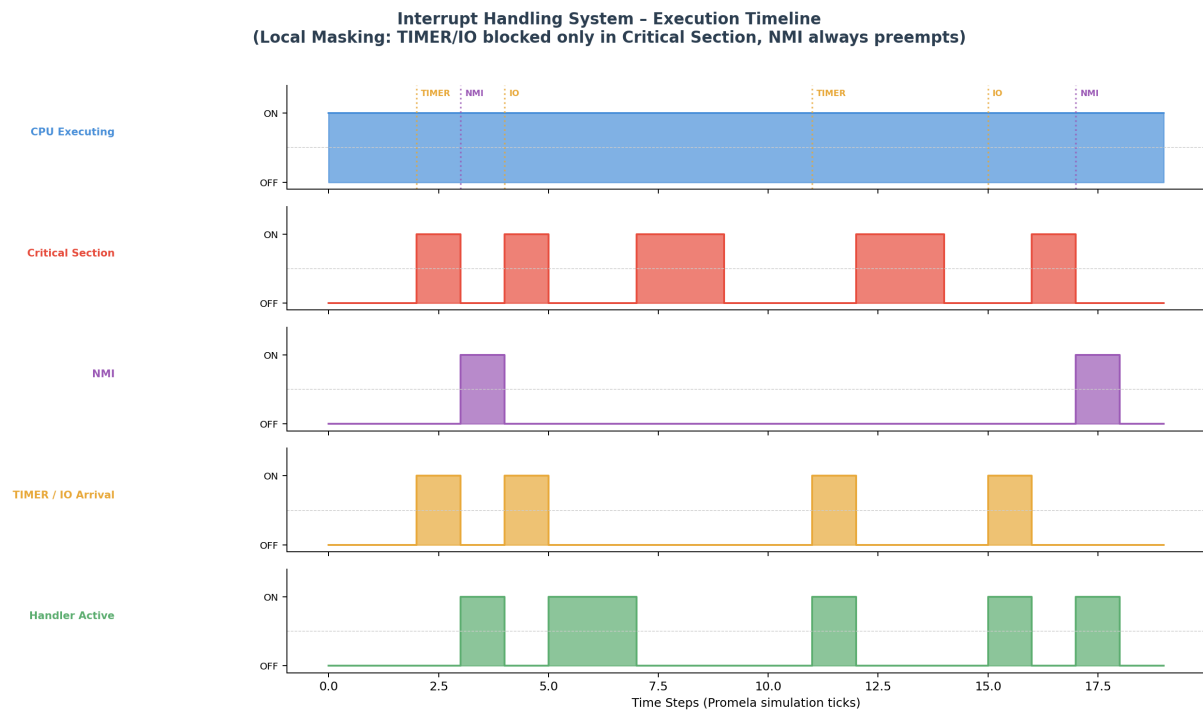


Figure 5: Interrupt Handling System — Execution Timeline

5.4 Key Observations

- **Liveness properties** (P1, P2, P5, P6, P8) required SPIN's `-a` flag for acceptance-cycle detection. Verification confirms there is no reachable cycle in which an interrupt remains permanently unhandled — ruling out starvation in all possible executions.
- **Safety properties** (P3, P4, P7, P9) are checked by reachability analysis. Their smaller state spaces reflect that a violation requires only a short execution path to witness.
- **Atomicity** of the dispatch sequence is essential for P8. Without `atomic{...}`, a valid process interleaving could allow an interrupt to slip in between the mask-clear and the channel-send, falsifying both properties.
- **Bounded channels** keep the state space finite and representable while faithfully modelling real hardware FIFOs.
- **Model decomposition** was effective: the largest model explored 1,340 states, well within SPIN's practical limits for exhaustive verification.

6 Discussion

6.1 Modelling Trade-offs

Priority levels. Real interrupt controllers such as Intel's APIC support up to 256 priority levels. We modelled three levels to keep the state space tractable. The structural argument — that a high-priority guard on queue selection enforces preemption — generalises directly to any finite number of levels.

Device behaviour. Devices in our model generate a small fixed number of interrupts. In production systems devices may fire indefinitely. Handling this rigorously requires either: (a) fairness constraints under SPIN's `-f` flag, so that every continuously enabled guard is eventually selected; or (b) data abstraction, modelling only counts modulo a small bound. Our approach uses (b) implicitly by capping iteration counts.

Atomicity. We use `atomic{...}` sparingly and only where hardware guarantees indivisibility (e.g., the CPU atomically saves the program counter before jumping to the ISR entry point). Over-use would mask real concurrency bugs; under-use inflates the state space.

6.2 NMI Bypass Correctness

The NMI path in the CPU bypasses `interrupt_enable` entirely, directly mirroring the NMI pin behaviour on x86. Property P2 verifies this produces no livelock: the NMI is always eventually dispatched, and `nmi_pending` is always eventually cleared by the handler. Using a dedicated boolean flag rather than an in-band queue entry avoids any risk of the NMI being dropped due to queue overflow — matching how NMI lines are physically implemented.

6.3 Safety vs. Liveness Verification

Safety properties assert that “something bad never happens” and are verified by reachability: SPIN checks that no reachable state violates the invariant. Liveness properties assert that “something good eventually happens” and require acceptance-cycle detection in the product automaton of the model and the negated LTL formula’s Büchi automaton. This is why liveness properties consistently explore larger state spaces (see Table 1) and why they require the `-a` verification mode.

7 Future Work

While the current models successfully verify the core properties of interrupt handling, several extensions would bring the model closer to production operating system behaviour.

Modern servers run multiple CPUs sharing an Advanced Programmable Interrupt Controller (APIC). Extending the model with multiple CPU `proctype` instances — each with its own interrupt enable flag — would allow verification of interrupt affinity and Inter-Processor Interrupt (IPI) delivery. Key properties to verify would include: no two CPUs handle the same interrupt simultaneously, and load balancing eventually distributes interrupts across all active cores.

Production device drivers generate interrupts indefinitely. Modelling this rigorously requires SPIN fairness constraints (`-f` flag) combined with Promela progress labels. Under weak fairness, every continuously enabled guard is assumed to eventually be selected — allowing liveness properties to be verified without bounding device iteration counts. This removes a key simplification assumption at the cost of slightly more complex property specifications.

The Linux kernel since version 3.10 supports a “tickless” (dynamic tick) mode where the timer interrupt fires only when needed rather than at a fixed rate. Modelling this requires non-deterministic timer intervals and a priority queue rather than a FIFO for the IRQ channel. Properties to verify would include: the system always wakes up within a bounded delay when a timer event is due, and no timer event is ever silently dropped.

As the model grows with more devices and priority levels, full state-space exploration becomes expensive. SPIN supports partial-order reduction, which prunes equivalent interleavings and can reduce the state space by orders of magnitude. For models with multiple structurally identical devices, symmetry reduction could further compress the state space. Exploring the degree of compression achievable while preserving soundness would be a valuable study.

The current model is untimed — it verifies ordering properties but not timing bounds. Extending to a timed model using Uppaal (a timed-automata model checker) would enable verification of real-time constraints such as: “every interrupt is acknowledged within N clock cycles of generation.” This is particularly relevant for hard real-time systems in the automotive and aerospace domains where deadline violations are safety-critical.

8 Conclusion

This assignment demonstrated that Promela and SPIN provide an effective framework for modelling and verifying real operating system mechanisms. We designed three complementary models capturing the full interrupt handling pipeline — from asynchronous device signal through bounded FIFO queuing, CPU dispatch with interrupt masking, to handler acknowledgement — and formally verified ten LTL properties spanning liveness, safety, and mutual exclusion.

All ten properties were verified successfully. The following guarantees now hold for every reachable execution of the model:

Guarantee	Description
No lost interrupts	Every device-generated interrupt is eventually handled
No spurious interrupts	The handler never processes more events than were generated
Mask correctness	Interrupt masking correctly defers IRQs across critical sections
NMI bypass	Non-maskable interrupts always reach the handler
Priority enforcement	High-priority interrupts always preempt lower-priority ones
Clean termination	The system reaches quiescence once all events are processed

The exercise also surfaced practical modelling challenges: bounding device behaviour for tractability, choosing the right granularity of `atomic` blocks, and decomposing a complex system across multiple models to keep each state space manageable. These challenges reflect real difficulties in industrial formal verification, and the solutions applied here (data abstraction, careful atomicity, and concern separation) are standard practice in verified OS kernel development.

Appendix: File Listing

File	Type	Description
<code>interrupt_system.pml</code>	Promela	Full system model — P1–P4
<code>priority_preemption.pml</code>	Promela	Priority scheduling model — P5–P7
<code>masking_critical.pml</code>	Promela	Masking & critical section model — P8–P10
<code>fig1_architecture.py</code>	Python	System architecture diagram
<code>fig2_timeline.py</code>	Python	Execution timeline chart
<code>fig3_verification.py</code>	Python	Verification results bar chart
<code>report.tex</code>	L ^A T _E X	This report